



Week 1 Session 2

Introduction to Python

Discussion from last session

Programming languages can be categorised in various ways:

- Low level / high level
- Paradigm

Consider the following general-purpose languages and categorise them:

- Java, C++, C#, Rust, C

We can also categorise languages as Imperative or Declarative

- Find an example of each and explain why it is

Today's Aims:

Introduce Python

- A little history, language properties

Starting programming with Python

- be confident using `print()` and `input()` in python
- understand the different data types in python
- be able to convert between data types as appropriate
- use some basic string methods
- use arithmetic operators

‘I’ve already done this’

If you have studied Python before, you should still find these sessions useful – we need to start with the basics so everyone can catch up, but we will do things in a different way than you may have learned.

There are some more advanced tasks and other things for you to be doing while we cover the basics – check the ‘additional tasks’ folder in the repository.

Python

- A high-level programming language
 - meaning more like natural (spoken) language than machine code
- Considered to be easy to work with
 - less verbose (ie. less code) than other common languages
 - easy-to-read error messages
- Nowadays a common tool in many areas
 - data science
 - natural language processing
 - AI



Python: a brief history

- Over 30 years old
- Current version Python 3 about 15 years old
- The move from Python 2 to Python 3 was controversial
 - Why?
- Python 4 is not planned



Programming in Codespaces

Last session we forked the semester 1 repository

- Open a Codespace from the menu
- Navigate in the terminal to “week1.1/session2” folder
 - You can review linux commands from last session’s notes

Task: Hello World

It's a programming tradition to write some code which prints out 'Hello World' as your first program.

Open `1_hello_world.py`

Let's write it together.

Hints

- Recall some linux commands:
 - pwd – to find out which folder I am currently in
 - cd – to move to a different folder, eg. cd week_01
 - ls – to view folder contents
- Using Python:
 - Navigate to the folder where your code file exists
 - Use “python” – to run your code, eg. type

```
python 1_hello_world.py
```

Running Python programs

- Python is an interpreted language
- When we type `python 1_hello_world.py`
 - We are invoking the Python interpreter with the input source code file
- When we run the interpreter a separate application
 - the PVM (Python virtual machine):
 - Reads the code line by line
 - Compiles it to machine-level instructions (byte-code)
 - Executes it



Variables

A variable is a container (object) where you can store some data.

We give it a variable name and an initial value (using =)

* This is known as assignment

The value can change throughout the program.

```
name = "Amy"
```

```
date = "20-10-2024"
```

```
speed = 11.423
```

```
distance = 102374
```

Variables Continued

There are some rules about variable names:

- no spaces
- no symbols
- always start with a letter
- don't use reserved words (names of existing functions, for example 'input' or 'print')

In python, we usually use snake_case to write variable names - where you would have a space, use an _ underscore _ instead.

Python programming

- Python is an object-oriented language
 - All data within a Python program are objects
- It does not enforce object-oriented programming
 - Considered to be multi-paradigm
 - We can create Python programs as
 - Object-oriented
 - Procedural (which is what we are doing at present)
 - Functional



Variable type

If we assign a variable a value (in this case a floating point number)

```
speed = 11.423
```

we can later (re-)assign a different type (a character string)

```
speed = "fast"
```

This is because the variable 'speed' is an object – a container for the data

Python programming

- Python variables are dynamically typed
 - The variable can change type through assignment
 - (The converse is called static typing)
- Python variables are strongly typed
 - The type of a variable cannot change in unexpected ways
 - (The converse is weak typing)



Printing With Variables

You can print the *value* of a variable very easily:

```
name = "Amy"  
print(name)      # This prints: Amy
```

Note that we added a comment to our code using #

- Everything after # is for information and ignored by the interpreter

Printing With Variables and Text

To include both text and the value of a variable, we can use f-strings. This makes the code and output easier to read.

```
name = "Amy"  
print( f"Hello {name}!" )      # This prints: Hello Amy!
```

Note

- The 'f' at the start of the printing statement
- That the curly brackets {.} enclose a variable - the value is printed

Task 1

- Add to your 'hello world' so that it prints out a name stored in a variable
- Make your program say hello to three different people by changing the value and printing again.

User Input

Instead of hard-coding variables - having their value assigned in the code
- we can ask people to input values:

```
name = input("What is your name?")
```

The input() function reads from the keyboard and assigns the text to the variable 'name'

Let's adapt our code to do this!

Task 2

- Adapt your 'hello world' program to have a conversation
 - Ask 3 questions, and reply to your user in a way which uses their response
 - For example:

```
What's your name? Amy
Oh cool, I had a friend at school called Amy
Where did you grow up? York
I hear there's a nice museum in York!
What's your favourite animal? echidnas
I personally prefer giraffes but echidnas are cool too I guess
```
- You can use your own questions and responses!



Different Types of Variable

Integers, strings and floats

Numbers

Now we can take input into the program, let's make a very basic calculator.

`2_calculator.py`

We will:

- Ask the person to enter two numbers
- Make another variable 'answer' which holds the result of adding them together
- Use f-strings to display both the numbers and the result in a meaningful way

Why didn't that work?

When we use `input()` Python stores the value as a data type called a string. (ie. character data)

Python doesn't recognise that the string is a number

- It is treating them as text
- This is a consequence of strong typing

We need to use a different data type.

- We will look at 3 basic data types today:

Data Types



String - any text, symbols or numbers inside inverted commas
("hello", "3.83.11", "01904")



Integer - whole numbers
(3, 974, -1)



Float - numbers with decimal points
(3.1415, -8.624, 3.0)

Fixing Our Calculator

To turn our strings into integers, we will convert them

- The Python built-in function `int()` converts a string to an integer

The easiest way to do this is to do this when we read the input:

```
num = int(input("Enter a number: "))
```

Task 3 – Common Errors

It's easy to make mistakes when programming – and it's important to start to understand how to find and fix them.

3_debugging.py contains several common errors I see students make.

You need to find and fix them all so that the code works.

User Error

- Try entering a letter instead of a number in any input which has an `int()` around it...

```
Enter your first number: no
Traceback (most recent call last):
  File "/workspaces/the-junk-drawer/example2.py", line 1, in <module>
    num = int(input("Enter your first number: "))
           ~~~~~^
ValueError: invalid literal for int() with base 10: 'no'
```

Writing Robust Code

As a user, you do not want the program you are using to do this.

As a user, you want programs to tell you what you did wrong and let you try again!

As a developer, it's your job to make things fail gracefully – this means we should think about:

- What might our user do wrong?
- What should the program do in response?
- How do we test that this happens?

Try... Except...

One of the easiest ways to protect your code is to use try ... except

```
try:
```

```
    num1 = int(input("Enter your number: "))
```

```
    num2 = int(input("Enter your number: "))
```

```
    answer = num1 + num2
```

```
    print(f"{num1} + {num2} = {answer}")
```

```
except:
```

```
    print("Please enter numbers only.")
```

This will prevent the program from continuing if there is an error.

Indentation and scope

You will see from the code example that inside “try:” we indent

- Indented code defines “scope”
- ie. The indented code is only in the scope of the “try” statement

Python recognises 2 forms of indentation

- <tab> space
- 4 spaces

We recommend only to use “4 spaces”

- <tab> can be redefined to be more or less space

Try... Except... Explained

What does it mean?

try: Run this code until it ends or there is an error

```
num1 = int(input("Enter your number: "))
```

```
num2 = int(input("Enter your number: "))
```

```
answer = num1 + num2
```

```
print(f"{num1} + {num2} = {answer}")
```

except: And if it causes an error, do this instead

```
print("Please enter numbers only.")
```



Maths in Python

Arithmetic Operators

Operators

Python has 7 math operators

+ - * /

** power

// floor division (ie. division rounded down to the smallest int)

% modulus (ie. remainder of integer division)

(We will see later there are many more built-in math functions)

Let's Get Some Practice!

For each of these maths expressions, calculate the answer using Python.
You can hard-code the values.

4_maths_practice.py

1. $(4 \times 8) \times 6$
2. $(2^3) / (8/3)$
3. $27^2 \times 19/4$

Hint: ^ doesn't work in Python

Formatting Float Outputs

You may have noticed that Python will print out many decimal places when working with floats.

This is often unhelpful when writing programs and makes it harder to read answers.

We can use f-strings to restrict `print()` to a fixed amount:

```
print(f"The answer is {answer:.3f}")
```

Modulo and Integer Division

In Python, we have two more operators:

// - integer division – gives you only the whole part of a division

$$8 // 3 = 2$$

% - modulo division – gives you only the remainder of a division

$$9 // 2 = 1$$

Modulo Task

5_module_intdiv.py

If you're feeling confident, you can start working on the task individually.

We can talk through it for anyone who wants some help.

Strings

One of the things which makes Python popular is its string handling

Python has a large set of built-in string functions.

This makes working with text very simple!

What is a string function?

A built-in function can be used for a specific task.

For example, we have already seen

- `int()` which converts a string to an int
- `input()` which takes keyboard input and produces a string

A string function is a function which processes a string in some way.

For example, `upper()`, which converts a string to upper case:

```
message = "hello"
```

```
print( message.upper() )
```

What can they do?

In '6_strings.py' there are lots of different string methods

- run the code and work out what each one does.
- Write a #comment to explain what it does.
 - Comments in Python begin with “#” and are ignored by the interpreter

Note:

- String functions do not change the original string
- they process it to produce a new string

Gradescope & Portfolios

You will be submitting your weekly portfolio tasks to Gradescope.

Sometimes these will be automatically graded by a script – for this to work, you need to upload the code with the right name and format.

To practice, you are going to complete a very short task and upload it to Gradescope to make sure you understand this process.

`7_gradescope.py`

End of Session Reminder

You need to add, commit and push your work to save it!

```
git add -A
```

```
git commit -m "end of session 2"
```

```
git push
```

Summary

We have:

- Introduced the Python programming language
- Used `input()` and `print()` for simple input/output
- Used f-strings for formatted output
- Looked at different data types in python and converting between them
- Use standard arithmetic operators
- Used some built-in string functions